

Section 8.3: Object-Oriented Design for LLD Interviews

Module 8 of 8 · SOLID Principles, Design Patterns & System Modelling

Executive Overview

Low-level design (LLD) interviews test whether you can translate a system description into clean, extensible object-oriented code. The evaluation covers three areas: SOLID principles (are your abstractions right?), design patterns (are you solving the right problem with the right pattern?), and state machine design (can you model lifecycle transitions correctly?). This section provides worked problems, comparison tables, and interview-ready solutions.

Part 1: SOLID Principles

1.1 Single Responsibility Principle (SRP)

Rule: A class should have exactly one reason to change.

```
 VIOLATION – four reasons to change
//
class OrderProcessor {
    public void process(Order order) {
        validate(order);           // Reason 1: validation rules change
        chargeCreditCard(order);  // Reason 2: payment provider changes
        sendEmail(order);         // Reason 3: email template changes
        updateInventory(order);    // Reason 4: inventory logic changes
    }
}

 CORRECT – each class has one reason to change
//
class OrderValidator      { void validate(Order o)           { /* validation
rules */ } }
class PaymentProcessor    { void charge(Payment p)          { /* payment
provider */ } }
class NotificationService { void sendConfirmation(Order o)   { /* email
templates */ } }
class InventoryManager    { void deduct(List<OrderItem> i)    { /* inventory
logic */ } }

// Orchestration responsibility lives in a separate use-case class
```

```

class PlaceOrderUseCase {
    void execute(Order order) {
        validator.validate(order);
        paymentProcessor.charge(order.payment());
        inventoryManager.deduct(order.items());
        notificationService.sendConfirmation(order);
    }
}

```

1.2 Open/Closed Principle (OCP)

Rule: Open for extension, closed for modification. New behaviour is added by adding code, not changing existing code.

```

// ❌ VIOLATION – every new discount type requires modifying this class
class DiscountCalculator {
    double calculate(Order order, String type) {
        if (type.equals("FIXED"))    return order.total() * 0.10;
        if (type.equals("SEASONAL")) return order.total() * 0.15;
        if (type.equals("LOYALTY"))  return order.total() * 0.20; // Must touch
class to add
        return 0;
    }
}

// ✅ CORRECT – new discount = new class; no existing code changes
interface DiscountStrategy {
    double calculate(Order order);
    boolean appliesTo(Order order);
}

class FixedDiscount implements DiscountStrategy {
    public double calculate(Order order) { return order.total() * 0.10; }
    public boolean appliesTo(Order order) { return true; }
}

class LoyaltyDiscount implements DiscountStrategy {
    public double calculate(Order order) { return order.total() * 0.20; }
    public boolean appliesTo(Order order) { return
order.customer().loyaltyYears() >= 2; }
}

class DiscountEngine {
    private final List<DiscountStrategy> strategies;

    double bestDiscount(Order order) {
        return strategies.stream()
            .filter(s -> s.appliesTo(order))
            .mapToDouble(s -> s.calculate(order))
            .max()
    }
}

```

```
        .orElse(0);  
    }  
}
```

1.3 Liskov Substitution Principle (LSP)

Rule: A subtype must be substitutable for its supertype without breaking the program's correctness.

The classic violation: Square extends Rectangle but breaks the width/height independence contract.

```
// ❌ VIOLATION  
class Rectangle {  
    protected double width, height;  
    void setWidth(double w) { this.width = w; }  
    void setHeight(double h) { this.height = h; }  
    double area() { return width * height; }  
}  
  
class Square extends Rectangle {  
    @Override void setWidth(double w) { this.width = w; this.height = w; } //  
Breaks contract  
    @Override void setHeight(double h) { this.width = h; this.height = h; }  
}  
  
// Client code that breaks:  
void resizeAndCheck(Rectangle r) {  
    r.setWidth(5);  
    r.setHeight(3);  
    assert r.area() == 15; // Passes for Rectangle; FAILS for Square  
}  
  
// ✅ CORRECT – separate types; shared behaviour via interface only where  
contracts match  
interface Shape { double area(); }  
class Rectangle implements Shape { ... } // independent width, height  
class Square implements Shape { ... } // single side length  
// No inheritance where contracts diverge
```

1.4 Interface Segregation Principle (ISP)

Rule: Clients should not be forced to depend on methods they do not use. Split fat interfaces into role-specific ones.

```
// ❌ VIOLATION – Developer is forced to implement deploy() it doesn't own  
interface Worker { void code(); void test(); void deploy(); void writeDocs(); }
```

```

class Developer implements Worker {
    public void code()      { /* ... */ }
    public void test()      { /* ... */ }
    public void deploy()    { throw new UnsupportedOperationException(); } //
Smell!
    public void writeDocs() { /* ... */ }
}

// ✓ CORRECT – role-based interfaces
interface Coder            { void code(); void test(); }
interface Deployer         { void deploy(); }
interface TechnicalWriter { void writeDocs(); }

class Developer implements Coder, TechnicalWriter { ... }
class DevOpsEngineer implements Coder, Deployer { ... }
class TechWriter implements TechnicalWriter { ... }

```

1.5 Dependency Inversion Principle (DIP)

Rule: High-level modules should not depend on low-level modules. Both should depend on abstractions.

```

// ✗ VIOLATION – OrderService hard-wired to MySQL
class OrderService {
    private MySQLOrderRepository repo = new MySQLOrderRepository(); // Direct
dependency

    void save(Order order) { repo.save(order); }
}
// Cannot test without MySQL; cannot swap to DynamoDB without editing this class

// ✓ CORRECT – both depend on interface
interface OrderRepository { void save(Order o); Order find(OrderId id); }

class OrderService {
    private final OrderRepository repo; // Abstraction
    OrderService(OrderRepository repo) { this.repo = repo; } // Injected

    void save(Order order) { repo.save(order); }
}

class MySQLOrderRepository implements OrderRepository { ... }
class DynamoOrderRepository implements OrderRepository { ... }
class InMemoryOrderRepository implements OrderRepository { ... } // For tests

```

Part 2: Core Design Patterns — When to Use Each

2.1 Comparison: Creational Patterns

Pattern	Use When	Avoid When
Factory Method	Object creation logic varies by type; caller should not know concrete class	Only one type; simple <code>new</code> suffices
Abstract Factory	Need to create families of related objects (UI toolkit: buttons + checkboxes + inputs)	Creating single unrelated objects
Builder	Object has many optional parameters; step-by-step construction needed	Object has ≤ 3 fields, all required
Singleton	Exactly one instance must exist globally (config, logger)	You want testability — Singleton couples to global state

```
// Builder — for complex optional configuration
class HttpClient {
    private final String baseUrl;
    private final int timeoutMs;
    private final int maxRetries;
    private final boolean followRedirects;
    private final Map<String, String> defaultHeaders;

    private HttpClient(Builder b) {
        this.baseUrl = b.baseUrl;
        this.timeoutMs = b.timeoutMs;
        this.maxRetries = b.maxRetries;
        this.followRedirects = b.followRedirects;
        this.defaultHeaders = Map.copyOf(b.defaultHeaders);
    }

    static class Builder {
        private final String baseUrl; // required
        private int timeoutMs = 5000;
        private int maxRetries = 3;
        private boolean followRedirects = true;
        private Map<String, String> defaultHeaders = new HashMap<>();

        Builder(String baseUrl) { this.baseUrl = baseUrl; }
        Builder timeout(int ms) { this.timeoutMs = ms; return this; }
        Builder retries(int n) { this.maxRetries = n; return this; }
        Builder noRedirects() { this.followRedirects = false; return
this; }
        Builder header(String k, String v) { this.defaultHeaders.put(k, v);
```

```

return this; }
    HttpClient build()                { return new HttpClient(this); }
}

HttpClient client = new HttpClient.Builder("https://api.example.com")
    .timeout(3000)
    .retries(5)
    .header("Authorization", "Bearer " + token)
    .build();

```

2.2 Comparison: Structural Patterns

Pattern	Use When	How It Differs From Others
Adapter	Incompatible interface; can't change source	Wraps an existing object; changes its interface
Decorator	Add behaviour dynamically at runtime	Wraps and delegates; caller uses same interface
Facade	Simplify access to a complex subsystem	Hides subsystem; doesn't add behaviour
Proxy	Control access (lazy init, auth, caching)	Same interface as real object; intercepts calls

```

// Adapter – third-party Stripe incompatible with internal PaymentProcessor
interface
interface PaymentProcessor {
    PaymentResult process(PaymentRequest req);
}

class StripeAdapter implements PaymentProcessor {
    private final Stripe stripe;

    public PaymentResult process(PaymentRequest req) {
        // Translate PaymentRequest → Stripe's ChargeParams
        ChargeParams params = ChargeParams.builder()
            .amount(req.amountCents())
            .currency(req.currency())
            .source(req.cardToken())
            .build();
        Charge charge = stripe.charges().create(params);
        // Translate ChargeResult → PaymentResult
        return new PaymentResult(charge.getId(), mapStatus(charge.getStatus()));
    }
}

```

```
// Decorator – add logging/retry without touching core logic
class RetryingPaymentProcessor implements PaymentProcessor {
    private final PaymentProcessor delegate;
    private final int maxAttempts;

    public PaymentResult process(PaymentRequest req) {
        for (int attempt = 1; attempt <= maxAttempts; attempt++) {
            try {
                return delegate.process(req);
            } catch (TransientException e) {
                if (attempt == maxAttempts) throw e;
                backoff(attempt);
            }
        }
        throw new IllegalStateException("unreachable");
    }
}
```

2.3 Comparison: Behavioral Patterns

Pattern	Use When	Core Mechanism
Strategy	Algorithm varies; want to swap at runtime	Interface + multiple implementations; injected
Observer	Event notification to unknown/ multiple listeners	Subject holds list of observers; notifies all
Command	Encapsulate an action; support undo, queue, log	Object wraps operation; execute/ undo methods
State	Object behaviour depends on internal state	Delegate to state objects; transitions explicit
Template Method	Algorithm skeleton fixed; steps vary	Abstract base class defines skeleton; subclasses fill steps

```
// Strategy – pluggable sorting
interface SortStrategy<T> {
    void sort(List<T> items, Comparator<T> comparator);
}

class QuickSort<T> implements SortStrategy<T> {
    public void sort(List<T> items, Comparator<T> c) { /* quicksort */ }
}

class MergeSort<T> implements SortStrategy<T> {
    public void sort(List<T> items, Comparator<T> c) { /* mergesort */ }
}
```

```

}

// Observer – event notification
interface EventListener<T> {
    void onEvent(T event);
}

class OrderEventBus {
    private final Map<Class<?>, List<EventListener<?>>> listeners = new
    HashMap<>();

    @SuppressWarnings("unchecked")
    public <T> void subscribe(Class<T> eventType, EventListener<T> listener) {
        listeners.computeIfAbsent(eventType, k -> new
        ArrayList<>()).add(listener);
    }

    @SuppressWarnings("unchecked")
    public <T> void publish(T event) {
        List<EventListener<?>> subs = listeners.getDefault(event.getClass(),
        List.of());
        for (EventListener<?> sub : subs) {
            ((EventListener<T>) sub).onEvent(event);
        }
    }
}

```

Part 3: Interview Design Problems

Problem 1 — Parking Lot System

Question: Design a thread-safe parking lot with multiple levels, multiple vehicle types, and multiple entry/exit points. How do you handle race conditions without deadlocks?

Core Model:

```

enum VehicleType { MOTORCYCLE, CAR, TRUCK }
enum SpotSize     { SMALL, MEDIUM, LARGE }

class Vehicle {
    private final String licensePlate;
    private final VehicleType type;
    // Motorcycle fits SMALL/MEDIUM/LARGE; Car fits MEDIUM/LARGE; Truck fits LARGE
    only
    public List<SpotSize> compatibleSizes() { ... }
}

class ParkingSpot {
    private final int id;

```



```

    private final SpotSize size;
    private final AtomicReference<String> occupantPlate = new
AtomicReference<>(null);

    public boolean occupy(String plate) {
        return occupantPlate.compareAndSet(null, plate); // CAS – atomic, no lock
    }

    public boolean vacate(String plate) {
        return occupantPlate.compareAndSet(plate, null);
    }

    public boolean isAvailable() { return occupantPlate.get() == null; }
}

class Level {
    private final int floor;
    private final List<ParkingSpot> spots;
    // Indexed by SpotSize for fast lookup
    private final Map<SpotSize, Queue<ParkingSpot>> availableBySize;

    public Optional<ParkingSpot> findAndOccupy(Vehicle vehicle) {
        for (SpotSize size : vehicle.compatibleSizes()) {
            Queue<ParkingSpot> candidates = availableBySize.get(size);
            ParkingSpot spot;
            while ((spot = candidates.peek()) != null) {
                if (spot.occupy(vehicle.licensePlate())) {
                    candidates.poll(); // Remove from available queue
                    return Optional.of(spot);
                }
                candidates.poll(); // Spot was taken by another thread; try next
            }
        }
        return Optional.empty();
    }
}

class ParkingLot {
    private final List<Level> levels;
    private final ConcurrentHashMap<String, ParkingSpot> activeTickets = new
ConcurrentHashMap<>();

    public Ticket park(Vehicle vehicle) {
        for (Level level : levels) {
            Optional<ParkingSpot> spot = level.findAndOccupy(vehicle);
            if (spot.isPresent()) {
                Ticket ticket = new Ticket(vehicle, spot.get(), Instant.now());
                activeTickets.put(ticket.id(), spot.get());
                return ticket;
            }
        }
        throw new ParkingLotFullException();
    }

    public Fee exit(String ticketId) {

```

```

        ParkingSpot spot = activeTickets.remove(ticketId);
        if (spot == null) throw new InvalidTicketException(ticketId);
        spot.vacate(ticketId); // Return to available
        return calculateFee(ticketId);
    }
}

```

Race condition solution: Use `compareAndSet` (CAS) on individual spots — no global lock needed. Two threads competing for the same spot: exactly one wins the CAS; the other retries. No deadlock because there is no lock ordering issue.

Spot state machine:

```

AVAILABLE —park()—► OCCUPIED —exit()—► AVAILABLE
AVAILABLE —reserve()—► RESERVED —confirm()—► OCCUPIED
RESERVED —timeout()—► AVAILABLE

```

Problem 2 — Elevator System

Question: Design an elevator system for a 20-floor building with 4 elevators. Minimize average wait time. How do you dispatch elevators efficiently?

Core Model:

```

enum Direction { UP, DOWN, IDLE }

class Elevator {
    private int currentFloor;
    private Direction direction;
    private final TreeSet<Integer> upStops;    // floors to visit going up
    private final TreeSet<Integer> downStops;  // floors to visit going down

    // SCAN algorithm: service all stops in current direction before reversing
    public void requestStop(int floor) {
        if (direction == UP || (direction == IDLE && floor >= currentFloor))
            upStops.add(floor);
        else
            downStops.add(floor);
    }

    public int nextFloor() {
        if (direction == UP && !upStops.isEmpty()) return upStops.first();
        if (direction == DOWN && !downStops.isEmpty()) return downStops.last();
        // Reverse direction
        if (!downStops.isEmpty()) { direction = DOWN; return downStops.last(); }
        if (!upStops.isEmpty()) { direction = UP; return upStops.first(); }
        direction = IDLE;
        return currentFloor;
    }
}

```

```

    }

    public int estimatedArrivalTime(int targetFloor) {
        // Rough estimate: count floors to traverse + existing stops
        int floorsToTraverse = Math.abs(targetFloor - currentFloor);
        int existingStops = (targetFloor > currentFloor ? upStops : downStops)
            .headSet(targetFloor).size();
        return (floorsToTraverse + existingStops) * FLOOR_TIME_MS;
    }
}

class ElevatorController {
    private final List<Elevator> elevators;

    // Dispatch: choose elevator with minimum estimated arrival time
    public void requestElevator(int floor, Direction intent) {
        Elevator best = elevators.stream()
            .filter(e -> isMovingToward(e, floor) || e.isIdle())
            .min(Comparator.comparingInt(e -> e.estimatedArrivalTime(floor)))
            .orElse(leastBusy());

        best.requestStop(floor);
    }

    private boolean isMovingToward(Elevator e, int floor) {
        return (e.direction() == UP && floor >= e.currentFloor()) ||
            (e.direction() == DOWN && floor <= e.currentFloor());
    }
}

```

Dispatch strategies comparison:

Strategy	Algorithm	Avg Wait	Complexity	Use When
Nearest Idle	Pick closest IDLE elevator	Low when idle available	Simple	Low/medium traffic
SCAN (Disk elevator)	Service all stops in direction before reversing	Good under load	Medium	Medium/high traffic
LOOK	Like SCAN but reverse at last request, not floor boundary	Better than SCAN	Medium	General purpose
Zoning	Assign floors to elevators statically	Predictable	Simple	Very high-rise buildings

Problem 3 — Vending Machine

Question: Design a vending machine that handles product selection, payment (coins + notes), dispensing, and change return. Handle invalid inputs gracefully.

State Machine Design:

```
enum VendingState { IDLE, ITEM_SELECTED, PAYMENT_RECEIVED, DISPENSING }

class VendingMachine {
    private VendingState state = IDLE;
    private Product selectedProduct;
    private Money insertedAmount = Money.ZERO;
    private final Inventory inventory;
    private final MoneyHandler moneyHandler;

    // State transition table
    public void selectItem(String productCode) {
        if (state != IDLE) throw new InvalidStateException("Cannot select item in
state: " + state);
        Product product = inventory.find(productCode)
            .orElseThrow(() -> new ProductNotFoundException(productCode));
        if (!inventory.isAvailable(productCode))
            throw new ProductOutOfStockException(productCode);
        selectedProduct = product;
        state = ITEM_SELECTED;
    }

    public void insertMoney(Money amount) {
        if (state != ITEM_SELECTED && state != PAYMENT_RECEIVED)
            throw new InvalidStateException("Select item before inserting money");
        insertedAmount = insertedAmount.add(amount);
        if (insertedAmount.isGreaterThanOrEqualTo(selectedProduct.price())) {
            state = PAYMENT_RECEIVED;
        }
    }

    public DispensingResult dispense() {
        if (state != PAYMENT_RECEIVED)
            throw new InvalidStateException("Insufficient payment");

        state = DISPENSING;
        inventory.decrement(selectedProduct.code());
        Money change = insertedAmount.subtract(selectedProduct.price());
        Money returnedChange = moneyHandler.makeChange(change);

        // Reset state
        Product dispensed = selectedProduct;
        selectedProduct = null;
        insertedAmount = Money.ZERO;
        state = IDLE;

        return new DispensingResult(dispensed, returnedChange);
    }
}
```

```

    }

    public Money cancel() {
        Money refund = insertedAmount;
        insertedAmount = Money.ZERO;
        selectedProduct = null;
        state = IDLE;
        return refund;
    }
}

```

State transition diagram:

```

IDLE --selectItem()--> ITEM_SELECTED --insertMoney() [amount < price]-->
ITEM_SELECTED                                --insertMoney() [amount ≥ price]-->
PAYMENT_RECEIVED                                --cancel()--> IDLE
PAYMENT_RECEIVED --dispense()--> DISPENSING --> IDLE
PAYMENT_RECEIVED --cancel()--> IDLE (refund)

```

Quick Reference

Principle	One-Line Rule	Common Violation
SRP	One reason to change	Class does validation + persistence + notification
OCP	Extend without modifying	<code>if/else</code> chain for types
LSP	Subtypes substitutable	Square extends Rectangle; breaks area contract
ISP	No forced unused methods	Fat interface with unimplemented methods
DIP	Depend on abstractions	<code>new MySQLRepo()</code> inside service class

Pattern	Category	Use It When
Factory	Creational	Caller should not know concrete type
Builder	Creational	

Pattern	Category	Use It When
		Many optional fields; step-by-step construction
Adapter	Structural	Incompatible interface; cannot change source
Decorator	Structural	Add behaviour at runtime; same interface
Strategy	Behavioral	Algorithm varies; swap at runtime
Observer	Behavioral	Notify multiple unknown listeners on event
State	Behavioral	Behaviour depends on current state